UNIT-I

Introduction & Elementary Data Structures: Introduction., Fundamentals of algorithm(Line count, Operation Count), Analysis of algorithms(Best, Average, Worst case), Asymptotic Notations(O, Ω,Θ). Recursive Algorithms, Analysis using Recurrence Relations, Master's Theorem.
Review of elementary datastructures - Graphs: BFS, DFS, Bi-Connected Components, Sets: representation, UNION, FIND operations

Algorithm:

An Algorithm is a finite sequence of instructions, each of which has a clear meaning and can be performed with a finite amount of effort in a finite length of time. No matter what the input values may be, an algorithm terminates after executing a finite number of instructions. In addition every algorithm must satisfy the followingcriteria:

- **Input:** there are zero or more quantities, which are externallysupplied;
- **Output:** at least one quantity isproduced
- **Definiteness:** each instruction must be clear andunambiguous;
- **Finiteness:** if we trace out the instructions of an algorithm, then for all cases the algorithm will terminate after a finite number ofsteps;
- **Effectiveness:** every instruction must be sufficiently basic that it can in principle be carried out by a person using only pencil and paper. It is not enough that each operation be definite, but it must also befeasible.

In formal computer science, one distinguishes between an algorithm, and a program. A program does not necessarily satisfy the fourth condition. One important example of such a program for a computer is its operating system, which never terminates (except for system crashes) but continues in a wait loop until more jobs areentered.

We represent algorithm using a pseudo language that is a combination of the constructs of a programming language together with informal Englishstatements.

Psuedo code for expressing algorithms:

Algorithm Specification: Algorithm can be described in three ways.

1. Natural language like English: When this way is choused care should be taken, weshould ensure that each & every statement isdefinite.

2. Graphic representation called flowchart: This method will work well when thealgorithm is small&simple.

3. Pseudo-code Method: In this method, we should typically describe algorithms as program, which resembles language like Pascal &algol.

Pseudo-Code Conventions:

1. Comments begin with // and continue until the end ofline.

2. Blocks are indicated with matching braces{and}.

3. An identifier begins with a letter. The data types of variables are not explicitlydeclared.

4. Compound data types can be formed with records. Here is anexample,

```
Node.Record
{
  data type – 1data-1;
        .
        .
        .
  data type – n data – n;
  node * link;
}
```

Here link is a pointer to the record type node. Individual data items of a record can be accessed with → and period.

5. Assignment of values to variables is done using the assignmentstatement.

<Variable>:= <expression>;

6. There are two Boolean values TRUE andFALSE.

→ LogicalOperators AND, OR,NOT
→Relational Operators <, <=,>,>=, =,!=

7. The following looping statements areemployed.

For, while and repeat-until

While Loop:

```
While < condition > do
{
        <statement-1>
                .
                .
                .

        <statement-n>
}
```

For Loop:

For variable: = value-1 to value-2 step step do

```
{
  <statement-1>
        .
        .
        .
<statement-n>
}
```

repeat-until:

```
repeat
        <statement-1>
```

<statement-n>
until<condition>

8. A conditional statement has the followingforms.

   → If <condition> then <statement>
   → If <condition> then <statement-1>
   Else <statement-1>

   Case statement:

   Case
   {
   : <condition-1>**:**<statement-1>
   .
   .
   .
   : <condition-n>**:**<statement-n>
   : else **:** <statement-n+1>
   }

9. Input and output are done using the instructions read &write.

10. There is only one type of procedure:
    Algorithm, the heading takes theform,

    Algorithm <Name> (<Parameter lists>)

    → As an example, the following algorithm fields & returns the maximum of „n" given numbers:

```
1.  AlgorithmMax(A,n)
2.  // A is an array of size n
3. {
4.  Result :=A[1];
5.  for I:= 2 to ndo
6.    if A[I] > Resultthen
7.        Result:=A[I];
8.   return Result;
9. }
```

    In this algorithm (named Max), A & n are procedure parameters. Result & I are Local variables.

Algorithm:

```
1. Algorithm selection sort(a,n)
2. // Sort the array a[1:n] into non-decreasing
order.3.{
4.      for I:=1 to ndo
5.      {
```

6. j:=I;

```
7.              for k:=i+1 to ndo
8.                      if (a[k]<a[j])
9.                      t:=a[I];
10.                     a[I]:=a[j];
11.                     a[j]:=t;
12.     }
13. }
```

Performance Analysis:

The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical, and the other experimental. In performance analysis we use analytical methods, while in performance measurement we conductexperiments.

Time Complexity:

The time needed by an algorithm expressed as a function of the size of a problem is called the *time complexity* of the algorithm. The time complexity of a program is the amount of computer time it needs to run to completion.

The limiting behavior of the complexity as size increases is called the asymptotic time complexity. It is the asymptotic complexity of an algorithm, which ultimately determines the size of problems that can be solved by the algorithm.

The Running time of a program

When solving a problem we are faced with a choice among algorithms. The basis for this can be any one of the following:

i. We would like an algorithm that is easy to understand codeanddebug.
ii. We would like an algorithm that makes efficient use of the computer‟s resources, especially, one that runs as fast aspossible.

Measuring the running time of a program

The running time of a program depends on factors such as:

1. The input to theprogram.
2. The quality of code generated by the compiler used to create theobject program.
3. The nature and speed of the instructions on the machine used to execute the program,
4. The time complexity of the algorithm underlyingtheprogram.

Statement	*S/e*	*Frequency*	*Total*
1. Algorithm Sum(a,n)	0	-	0
2. {	0	-	0
3. S=0.0;	1	1	1
4. for I=1 to n do	1	n+1	n+1
5. s=s+a[I];	1	n	n
6. return s;	1	1	1
7. }	0	-	0

The total time will be 2n+3

Space Complexity:

The space complexity of a program is the amount of memory it needs to run to completion. The space need by a program has the following components:

Instruction space: Instruction space is the space needed to store the compiled version of the program instructions.

Data space: Data space is the space needed to store all constant and variable values. Data space has two components:

- Space needed by constants and simple variablesinprogram.
- Space needed by dynamically allocated objects such as arrays andclass instances.

Environment stack space: The environment stack is used to save information needed to resume execution of partially completed functions.

Instruction Space: The amount of instructions space that is needed depends on factors such as:

- The compiler used to complete the program intomachinecode.
- The compiler options in effect at the timeofcompilation
- The targetcomputer.

The space requirement s(p) of any algorithm p may therefore be written as,

S(P) = c+ Sp(Instance characteristics)

Where „c‟ is a constant.

Example 2:

```
Algorithm sum(a,n)
{
    s=0.0;
    for I=1 to n do
    s= s+a[I];
    return s;
}
```

- The problem instances for this algorithm are characterized by n,the number of elements to be summed. The space needed d by „n‟ is one word, since it is of type integer.
- The space needed by „a‟a is the space needed by variables of tyepe array offloating point numbers.
- This is atleast „n‟ words, since „a‟ must be large enough to hold the „n‟ elements tobe summed.
- So,we obtain Ssum(n)>=(n+s)

[n for a[],one each for n,I a&s]

Complexity of Algorithms

The complexity of an algorithm M is the function f(n) which gives the running time and/or storage space requirement of the algorithm in terms of the size „n‟ of the input data. Mostly, the storage space required by an algorithm is simply a multiple of the data size „n‟. Complexity shall refer to the running time ofthealgorithm.

The function f(n), gives the running time of an algorithm, depends not only on the size „n‟ of the input data but also on the particular data. The complexity function f(n) for certain casesare:

1. Best Case : The minimum possible value of f(n) is called the bestcase.
2. Average Case : The expected value off(n).

3. Worst Case : The maximum value of f(n) for any key possibleinput.

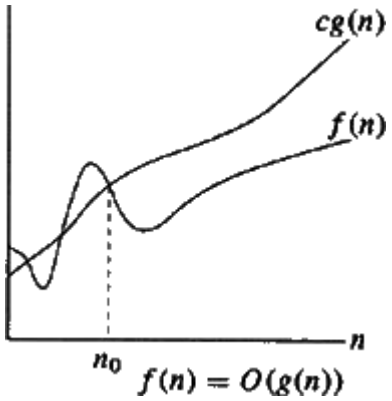
Asymptotic Notations:

The following notations are commonly use notations in performance analysis and used to characterize the complexity of an algorithm:

1. Big–OH(O)
2. Big–OMEGA(Ω),
3. Big–THETA (***Θ***)and
4. Little–OH(o)

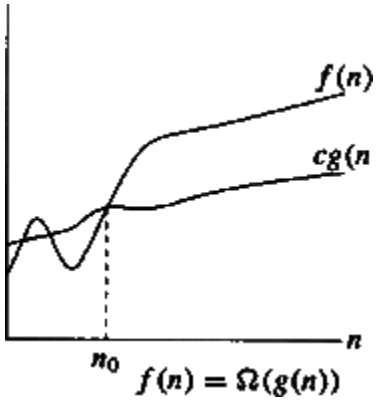
Big–OH O (Upper Bound)

f(n) = O(g(n)), (pronounced order of or big oh), says that the growth rate of f(n) is less than or equal (≤) that of g(n).



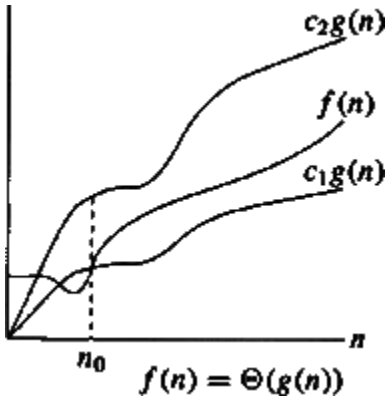
Big–OMEGA Ω (Lower Bound)

f(n) = Ω ***(g(n))*** (pronounced omega), says that the growth rate of f(n) is greater than or equal to (≥) that of g(n).



Big–THETA Θ (Same order)

f(n) = Θ (g(n)) (pronounced theta), says that the growth rate of f(n) equals (=) the growth rate of g(n) [if f(n) = O(g(n)) and T(n) = ***Θ*** (g(n)].



little-o notation

Definition: A theoretical measure of the execution of an *algorithm*, usually the time or memory needed, given the problem size n, which is usually the number of items. Informally, saying some equation f(n) = o(g(n)) means f(n) becomes insignificant relative to g(n) as n approaches infinity. The notation is read, "f of n is little oh of g of n".

Formal Definition: f(n) = o(g(n)) means for all c > 0 there exists some k > 0 such that $0 \leq f(n) < cg(n)$ for all $n \geq k$. The value of k must not depend on n, but may depend on c.

Different time complexities

Suppose „M" is an algorithm, and suppose „n" is the size of the input data. Clearly the complexity f(n) of M increases as n increases. It is usually the rate of increase of f(n) we want to examine. This is usually done by comparing f(n) with some standard functions. The most common computing times are:

O(1), O(log2 n), O(n), O(n. log2 n), O(n^2), O(n^3), O(2^n), n! and n^n

Classification of Algorithms

If „n" is the number of data items to be processed or degree of polynomial or the size of the file to be sorted or searched or the number of nodes in a graph etc.

1	Next instructions of most programs are executed once or at most only a few times. If all the instructions of a program have this property, we say that its running time is aconstant.
Logn	When the running time of a program is logarithmic, the program gets slightly slower as n grows. This running time commonly occurs in programs that solve a big problem by transforming it into a smaller problem, cutting the size by some constant fraction., When n is a million, log n is a doubled. Whenever n doubles, log n increases by a constant, but log n does not double until n increases ton^2.
n	Whentherunningtimeofaprogramislinear,itisgenerallythecasethata

small amount of processing is done on each input element. This is the optimal situation for an algorithm that must process n inputs.

n log n This running time arises for algorithms that solve a problem by breaking it up into smaller sub-problems, solving then independently, and then combining the solutions. When n doubles, the running time more than doubles.

n^2 When the running time of an algorithm is quadratic, it is practical for use only on relatively small problems. Quadratic running times typically arise in algorithms that process all pairs of data items (perhaps in a double nested loop) whenever n doubles, the running time increasesfourfold.

n^3 Similarly, an algorithm that process triples of data items (perhaps in a triple–nested loop) has a cubic running time and is practical for use only on small problems. Whenever n doubles, the running time increases eightfold.

2^n Few algorithms with exponential running time are likely to be appropriate for practical use, such algorithms arise naturally as "brute–force" solutions to problems. Whenever n doubles, the runningtimesquares.

Numerical Comparison of Different Algorithms

The execution time for six of the typical functions is given below:

n	log2 n	n*log2n	n^2	n^3	2^n
1	0	0	1	1	2
2	1	2	4	8	4
4	2	8	16	64	16
8	3	24	64	512	256
16	4	64	256	4096	65,536
32	5	160	1024	32,768	4,294,967,296
64	6	384	4096	2,62,144	Note 1
128	7	896	16,384	2,097,152	Note 2
256	8	2048	65,536	1,677,216	????????

Note1: The value here is approximately the number of machine instructions executed by a 1 gigaflop computer in 5000 years.

Randomized algorithms:

An algorithm that uses random numbers to decide what to do next anywhere in its logic is called Randomized Algorithm. For example, in Randomized Quick Sort, we use random number to pick the next pivot (or we randomly shuffle the array). Quicksort is a familiar, commonly used algorithm in which randomness can be useful. Any deterministic version of this algorithm requires O(n2) time to sort n numbers for some well-defined class of degenerate inputs (such as an already sorted array), with the specific class of inputs that generate this behavior defined by the protocol for pivot selection. However, if the algorithm selects pivot elements uniformly at random, it has a provably high probability of finishing in O(n log n) time regardless of the characteristics of the input. Typically, this randomness is used to reduce time complexity or space complexity in other standardalgorithms.

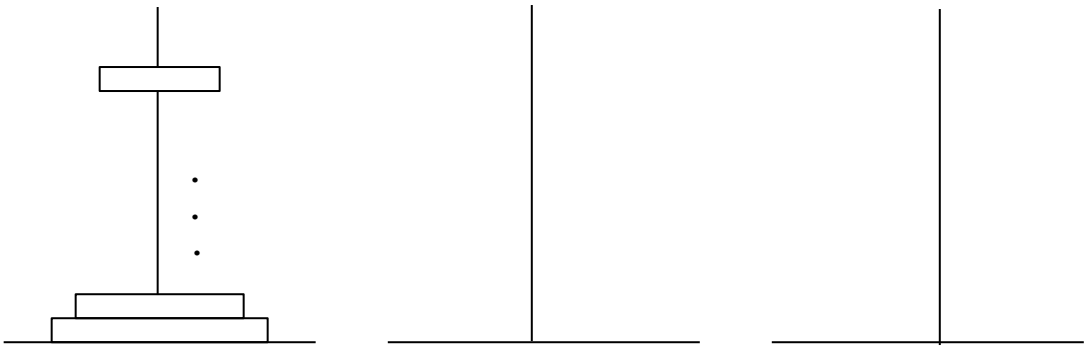
```
12.     }
13. }
```

Recursive Algorithms:

- A Recursive function is a function that is defined in terms of itself.
- Similarly, an algorithm is said to be recursive if the same algorithm is invoked in the body.
- An algorithm that calls itself is Direct Recursive.
- Algorithm „A" is said to be Indirect Recursive if it calls another algorithm which in turns calls „A".
- The Recursive mechanism, are externally powerful, but even more importantly, many times they can express an otherwise complex process very clearly. Or these reasons we introduce recursion here.
- The following 2 examples show how to develop a recursive algorithms.

  → In the first, we consider the Towers of Hanoi problem, and in the second, we generate all possible permutations of a list of characters.

1. **Towers of Hanoi:**



- It is Fashioned after the ancient tower of Brahma ritual.
- According to legend, at the time the world was created, there was a diamond tower (labeled A) with 64 golden disks.
- The disks were of decreasing size and were stacked on the tower in decreasing order of size bottom to top.
- Besides these tower there were two other diamond towers(labeled B & C)
- Since the time of creation, Brehman priests have been attempting to move the disks from tower A to tower B using tower C, for intermediate storage.
- As the disks are very heavy, they can be moved only one at a time.
- In addition, at no time can a disk be on top of a smaller disk.
- According to legend, the world will come to an end when the priest have completed this task.

- A very elegant solution results from the use of recursion.
- Assume that the number of disks is „n".
- To get the largest disk to the bottom of tower B, we move the remaining „n-1" disks to tower C and then move the largest to tower B.
- Now we are left with the tasks of moving the disks from tower C to B.
- To do this, we have tower A and B available.
- The fact, that towers B has a disk on it can be ignored as the disks larger than the disks being moved from tower C and so any disk scan be placed on top of it.

Algorithm:

```
1.  Algorithm TowersofHanoi(n,x,y,z)
2. //Move the top „n" disks from tower x to tower y.
3. {

        .
        .
        .

4. if(n>=1) then
5. {
6.        TowersofHanoi(n-1,x,z,y);
7.        Write("move top disk from tower " X ,"to top of tower " ,Y);
8.   Towersofhanoi(n-1,z,y,x);
9.   }
10. }
```

2. Permutation Generator:

- Given a set of n>=1elements, the problem is to print all possible permutations of this set.
- For example, if the set is {a,b,c} ,then the set of permutation is,

  { (a,b,c),(a,c,b),(b,a,c),(b,c,a),(c,a,b),(c,b,a)}
- It is easy to see that given „n" elements there are n! different permutations.
- A simple algorithm can be obtained by looking at the case of 4 statement(a,b,c,d)
- The Answer can be constructed by writing

1. a followed by all the permutations of (b,c,d)
2. b followed by all the permutations of(a,c,d)
3. c followed by all the permutations of (a,b,d)
4. d followed by all the permutations of (a,b,c)

Algorithm:

Algorithm perm(a,k,n)

Graphs

The Graph ADT Introduction

Definition

Graph representation

Elementary graph operations BFS, DFS

Introduction to Graphs

Graph is a non linear data structure; A map is a well-known example of a graph. In a map various connections are made between the cities. The cities are connected via roads, railway lines and aerial network. We can assume that the graph is the interconnection of cities by roads. Euler used graph theory to solve Seven Bridges of Königsberg problem. Is there a possible way to traverse every bridge exactly once – Euler Tour

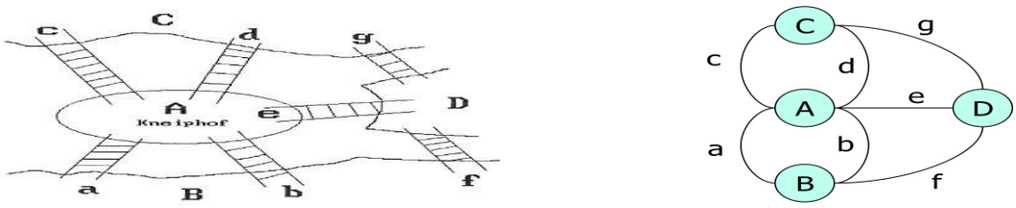


Figure: Section of the river Pregal in Koenigsberg and Euler's graph.

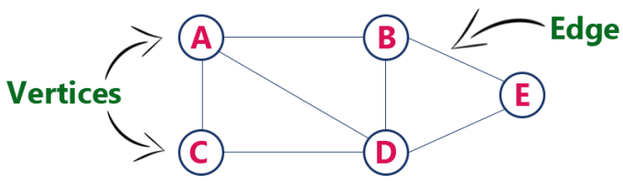
Defining the degree of a vertex to be the number of edges incident to it, Euler showed that there is a walk starting at any vertex, going through each edge exactly once and terminating at the start vertex iff the degree of each, vertex is even. A walk which does this is called Eulerian. There is no Eulerian walk for the Koenigsberg bridge problem as all four vertices are of odd degree.

A graph contains a set of points known as nodes (or vertices) and set of links known as edges (or Arcs) which connects the vertices.

A graph is defined as Graph is a collection of vertices and arcs which connects vertices in the graph. A graph G is represented as G = (V , E), where V is set of vertices and E is set of edges.

Example: graph G can be defined as G = (V , E) Where V = {A,B,C,D,E} and

E = {(A,B),(A,C)(A,D),(B,D),(C,D),(B,E),(E,D)}. This is a graph with 5 vertices and 6 edges.



Graph Terminology

1. **Vertex** : An individual data element of a graph is called as Vertex. Vertex is also known as node. In above example graph, A, B, C, D & E are known as vertices.

2. **Edge** : An edge is a connecting link between two vertices. Edge is also known as Arc. An edge is represented as (starting Vertex, ending Vertex).

In above graph, the link between vertices A and B is represented as (A,B).

Edges are three types:

1. Undirected Edge - An undirected edge is a bidirectional edge. If there is an undirected edge between vertices A and B then edge (A , B) is equal to edge (B , A).

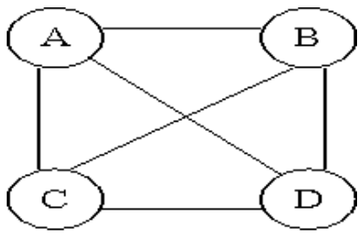
2. Directed Edge - A directed edge is a unidirectional edge. If there is a directed edge between vertices A and B then edge (A , B) is not equal to edge (B , A).

3. Weighted Edge - A weighted edge is an edge with cost on it.

Types of Graphs

1. Undirected Graph

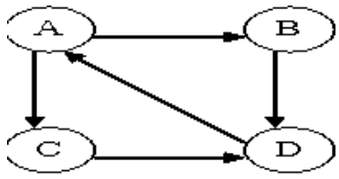
A graph with only undirected edges is said to be undirected graph.



Undirected Graph.

2. Directed Graph

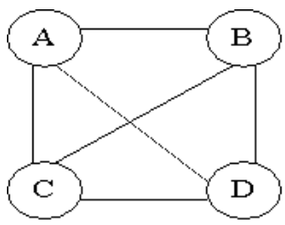
A graph with only directed edges is said to be directed graph.



Directed Graph.

3. Complete Graph

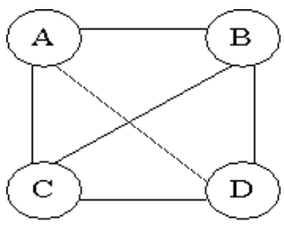
A graph in which any V node is adjacent to all other nodes present in the graph is known as a complete graph. An undirected graph contains the edges that are equal to edges = n(n-1)/2 where n is the number of vertices present in the graph. The following figure shows a complete graph.



A complete graph.

4. Regular Graph

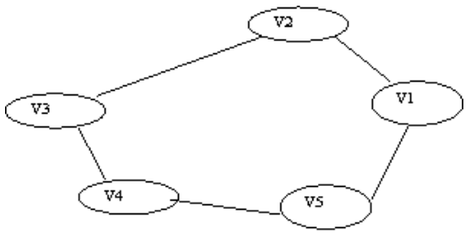
Regular graph is the graph in which nodes are adjacent to each other, i.e., each node is accessible from any other node.



A regular graph

5. Cycle Graph

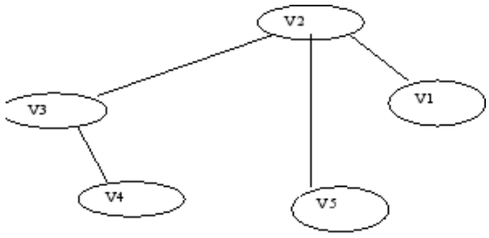
A graph having cycle is called cycle graph. In this case the first and last nodes are the same. A closed simple path is a cycle.



A cycle graph

6. Acyclic Graph

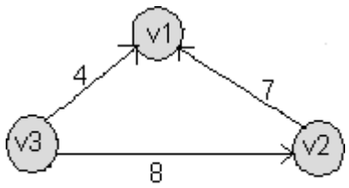
A graph without cycle is called acyclic graphs.



A acyclic graph

7. Weighted Graph

A graph is said to be weighted if there are some non negative value assigned to each edges of the graph. The value is equal to the length between two vertices. Weighted graph is also called a network.



A weighted graph

Outgoing Edge

A directed edge is said to be outgoing edge on its orign vertex.

Incoming Edge

A directed edge is said to be incoming edge on its destination vertex.

Degree

Total number of edges connected to a vertex is said to be degree of that vertex.

Indegree

Total number of incoming edges connected to a vertex is said to be indegree of that vertex.

Outdegree

Total number of outgoing edges connected to a vertex is said to be outdegree of that vertex.

Parallel edges or Multiple edges

If there are two undirected edges to have the same end vertices, and for two directed edges to have the same origin and the same destination. Such edges are called parallel edges or multiple edges.

Self-loop

An edge (undirected or directed) is a self-loop if its two endpoints coincide.

Simple Graph

A graph is said to be simple if there are no parallel and self-loop edges.

Adjacent nodes

When there is an edge from one node to another then these nodes are called adjacent nodes.

Incidence

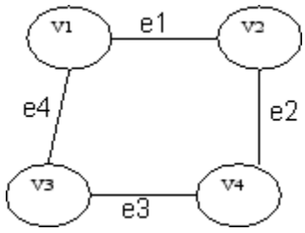
In an undirected graph the edge between v1 and v2 is incident on node v1 and v2.

Walk

A walk is defined as a finite alternating sequence of vertices and edges, beginning and ending with vertices, such that each edge is incident with the vertices preceding and following it.

Closed walk

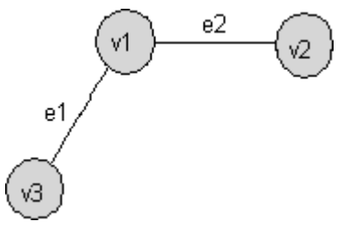
A walk which is to begin and end at the same vertex is called close walk. Otherwise it is an open walk.



If e1,e2,e3,and e4 be the edges of pair of vertices (v1,v2),(v2,v4),(v4,v3) and (v3,v1) respectively ,then v1 e1 v2 e2 v4 e3 v3 e4 v1 be its closed walk or circuit.

Path

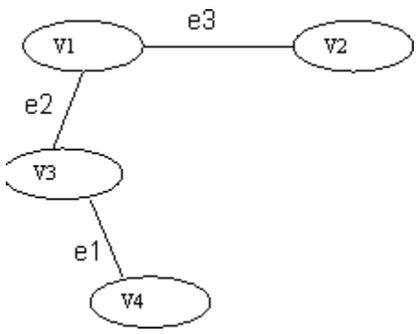
A open walk in which no vertex appears more than once is called a path.



If e1 and e2 be the two edges between the pair of vertices (v1,v3) and (v1,v2) respectively, then v3 e1 v1 e2 v2 be its path.

Length of a path

The number of edges in a path is called the length of that path. In the following, the length of the path is 3.

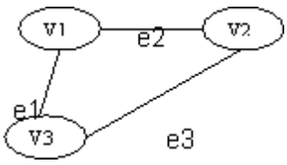


An open walk Graph

Circuit

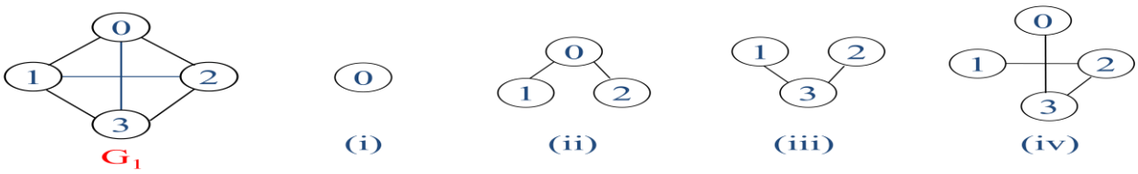
A closed walk in which no vertex (except the initial and the final vertex) appears more than once is called a circuit.

A circuit having three vertices and three edges.



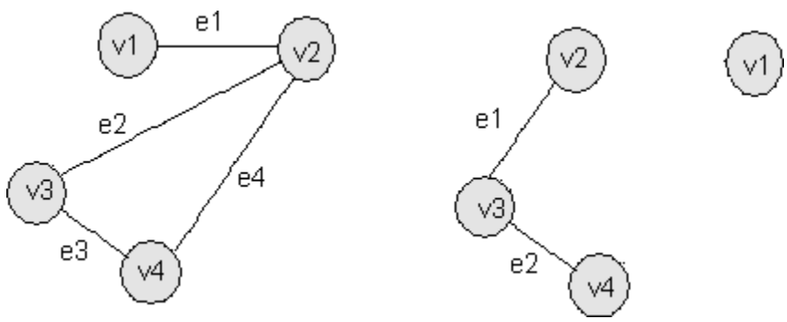
Sub Graph

A graph S is said to be a sub graph of a graph G if all the vertices and all the edges of S are in G, and each edge of S has the same end vertices in S as in G. A subgraph of G is a graph G' such that V(G') ⊆ V(G) and E(G') ⊆ E(G)



Connected Graph

A graph G is said to be connected if there is at least one path between every pair of vertices in G. Otherwise,G is disconnected.



A connected graph G A disconnected graph G

This graph is disconnected because the vertex v1 is not connected with the other vertices of the graph.

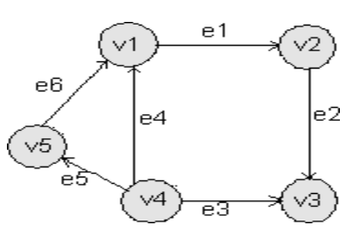
Degree

In an undirected graph, the number of edges connected to a node is called the degree of that node or the degree of a node is the number of edges incident on it.

In the above graph, degree of vertex v1 is 1, degree of vertex v2 is 3, degree of v3 and v4 is 2 in a connected graph.

Indegree

The indegree of a node is the number of edges connecting to that node or in other words edges incident to it.



In the above graph,the indegree of vertices v1, v3 is 2, indegree of vertices v2, v5 is 1 and indegree of v4 is zero.

Outdegree

The outdegree of a node (or vertex) is the number of edges going outside from that node or in other words the

ADT of Graph:

Structure Graph is

objects: a nonempty set of vertices and a set of undirected edges, where each edge is a pair of vertices

functions: for all *graph* ∈ *Graph*, *v*, v_1 and v_2 ∈ *Vertices*

Graph Create()::=return an empty graph

Graph InsertVertex(*graph*, *v*)::= return a graph with *v* inserted. *v* has no edge.

Graph InsertEdge(*graph*, *v1*,*v2*)::= return a graph with new edge between *v1* and *v2*

Graph DeleteVertex(*graph*, *v*)::= return a graph in which *v* and all edges incident to it are removed

Graph DeleteEdge(*graph*, *v1*, *v2*)::=return a graph in which the edge (*v1*, *v2*) is removed

Boolean IsEmpty(*graph*)::= if (*graph*==*empty graph*) return TRUE else return FALSE

List Adjacent(*graph*,*v*)::= return a list of all vertices that are adjacent to *v*

Graph Representations

Graph data structure is represented using following representations

1. **Adjacency Matrix**
2. **Adjacency List**
3. **Adjacency Multilists**

1.Adjacency Matrix

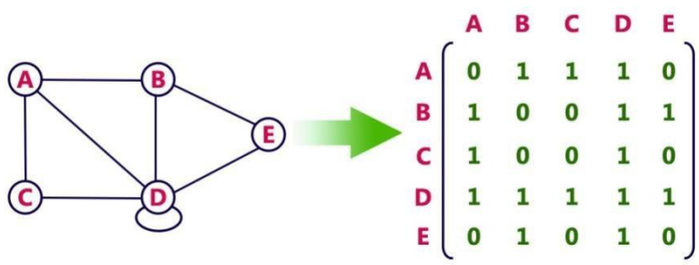
In this representation, graph can be represented using a matrix of size total number of vertices by total number of vertices; means if a graph with 4 vertices can be represented using a matrix of 4X4 size.

In this matrix, rows and columns both represent vertices.

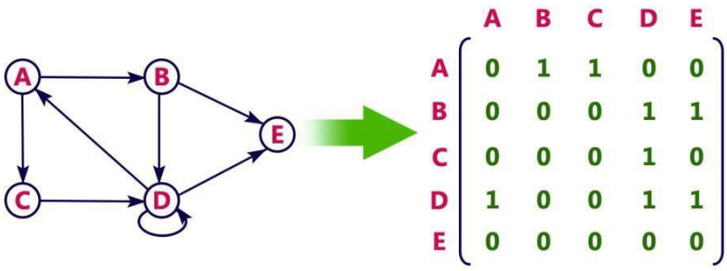
This matrix is filled with either 1 or 0. Here, 1 represents there is an edge from row vertex to column vertex and 0 represents there is no edge from row vertex to column vertex.

Adjacency Matrix : let G = (V, E) with n vertices, n ≥ 1. The adjacency matrix of G is a 2-dimensional n × n matrix, A, A(i, j) = 1 iff $(v_i, v_j) \in E(G)$ ($\langle v_i, v_j \rangle$ for a diagraph), A(i, j) = 0 otherwise.

example : for undirected graph



For a Directed graph



The adjacency matrix for an undirected graph is symmetric; the adjacency matrix for a digraph need not be symmetric.

Merits of Adjacency Matrix:

From the adjacency matrix, to determine the connection of vertices is easy

The degree of a vertex is $\sum_{j=0}^{n-1} adj\_mat[i][j]$

For a digraph, the row sum is the out_degree, while the column sum is the in_degree

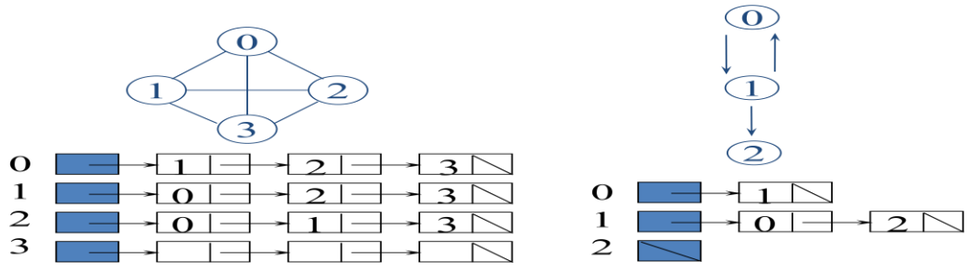
$$ind(vi) = \sum_{j=0}^{n-1} A[j,i] \qquad outd(vi) = \sum_{j=0}^{n-1} A[i,j]$$

The space needed to represent a graph using adjacency matrix is n^2 bits. To identify the edges in a graph, adjacency matrices will require at least $O(n^2)$ time.

2. Adjacency List

In this representation, every vertex of graph contains list of its adjacent vertices. The n rows of the adjacency matrix are represented as n chains. The nodes in chain I represent the vertices that are adjacent to vertex i.

It can be represented in two forms. In one form, array is used to store n vertices and chain is used to store its adjacencies. Example:

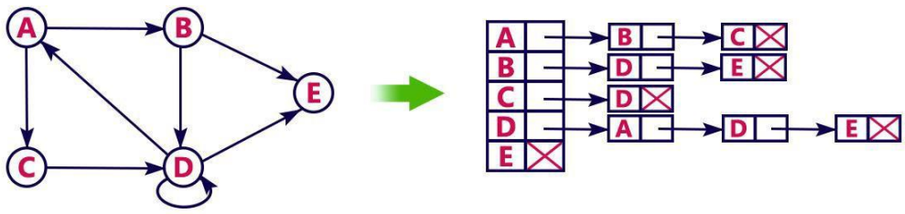


So that we can access the adjacency list for any vertex in O(1) time. Adjlist[i] is a pointer to to first node in the adjacency list for vertex i. Structure is

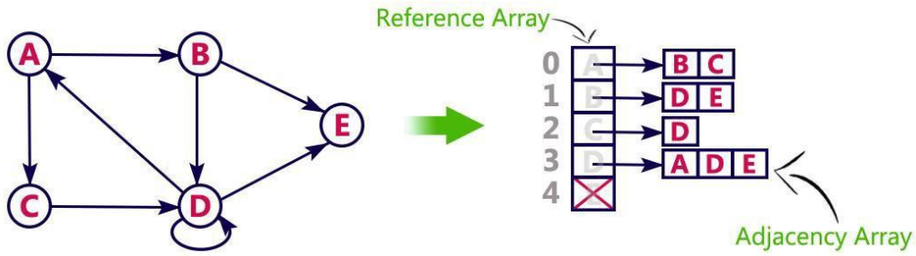
```
#define MAX_VERTICES 50
typedef struct node *node_pointer;
typedef struct node {
   int vertex;
   struct node *link;
};
node_pointer graph[MAX_VERTICES];
int n=0; /* vertices currently in use */
```

Another type of representation is given below.

example: consider the following directed graph representation implemented using linked list

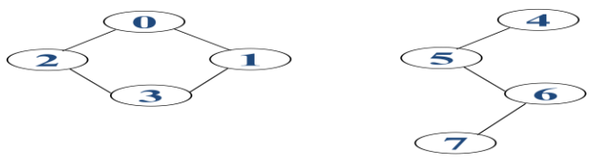

A
B
C
D
E
A
B
C
D
E
B
D
D
A
C
E
D
E

This representation can also be implemented using array



Sequential representation of adjacency list is

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
9	11	13	15	17	18	20	22	23	2	1	3	0	0	3	1	2	5	6	4	5	7	6

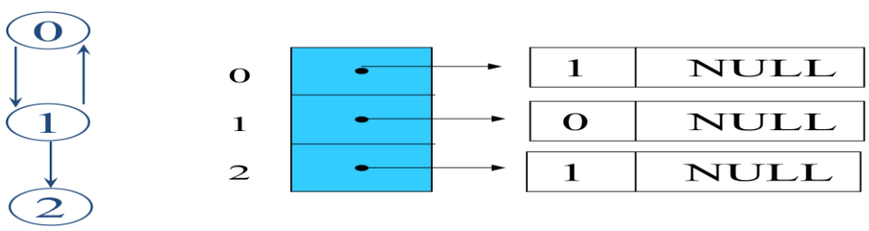


Graph

Instead of chains, we can use sequential representation into an integer array with size n+2e+1. For 0<=i<n, Array[i] gives starting point of the list for vertex I, and array[n] is set to n+2e+1. The adjacent vertices of node I are stored sequentially from array[i].

For an undirected graph with n vertices and e edges, linked adjacency list requires an array of size n and 2e chain nodes. For a directed graph, the number of list nodes is only e. the out degree of any vertex may be determined by counting the number of nodes in its adjacency list. To find in-degree of vertex v, we have to traverse complete list.

To avoid this, inverse adjacency list is used which contain in-degree.



Determine in-degree of a vertex in a fast way.

3. Adjacency Multilists

In the adjacency-list representation of an undirected graph each edge (u, v) is represented by two entries one on the list for u and the other on tht list for v. As we shall see in some situations it is necessary to be able to determin ie ~ nd enty for a particular edge and mark that edg as having been examined. This can be accomplished easily if the adjacency lists are actually maintained as multilists (i.e., lists in which nodes may be shared among several lists). For each edge there will be exactly one node but this node will be in two lists (i.e. the adjacency lists for each of the two nodes to which it is incident).

For adjacency multilists, node structure is

```
typedef struct edge *edge_pointer;
typedef struct edge {
   short int marked;
   int vertex1, vertex2;
   edge_pointer path1, path2;
};
edge_pointer graph[MAX_VERTICES];
```

marked	vertex1	vertex2	path1	path2

Lists: vertex 0: N0->N1->N2, vertex 1: N0->N3->N4
vertex 2: N1->N3->N5, vertex 3: N2->N4->N5

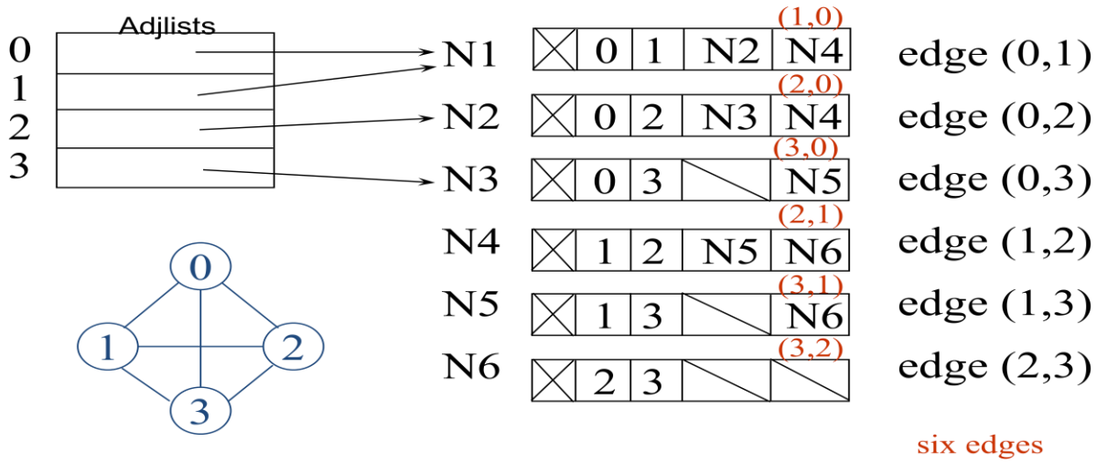
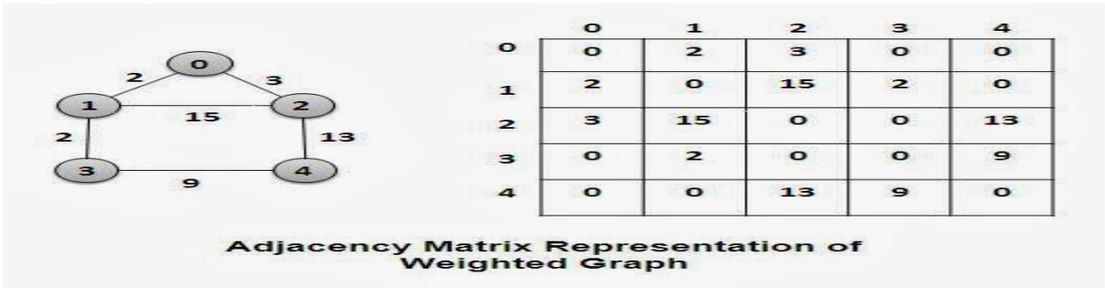


Figure: Adjacency multilists for given graph

4. Weighted edges

In many applications the edges of a graph have weights assigned to them. These weights may represent the distance from one vertex to another or the cost of going from one; vertex to an adjacent vertex In these applications the adjacency matrix entries A [i][j] would keep this information too. When adjacency lists are used the weight information may be kept in the list'nodes by including an additional field weight. A graph with weighted edges is called a network.



ELEMENTARY GRAPH OPERATIONS

Given a graph G = (V E) and a vertex v in V(G) we wish to visit all vertices in G that are reachable from v (i.e., all vertices that are connected to v). We shall look at two ways of doing this: depth-first search and breadth-first search. Although these methods work on both directed and undirected graphs the following discussion assumes that the graphs are undirected.

Depth-First Search

- Begin the search by visiting the start vertex v
  - If v has an unvisited neighbor, traverse it recursively
  - Otherwise, backtrack
- Time complexity
  - Adjacency list: O(|E|)
  - Adjacency matrix: $O(|V|^2)$

We begin by visiting the start vertex v. Next an unvisited vertex w adjacent to v is selected, and a depth-first search from w is initiated. When a vertex u is reached such that all its adjacent vertices have been visited, we back up to the last vertex visited that has an unvisited vertex w adjacent to it and initiate a depth-first search from w. The search terminates when no unvisited vertex can be reached from any of the visited vertices.

DFS traversal of a graph, produces a **spanning tree** as final result. **Spanning Tree** is a graph without any loops. We use **Stack data structure** with maximum size of total number of vertices in the graph to implement DFS

traversal of a graph.

We use the following steps to implement DFS traversal...

Step 1: Define a Stack of size total number of vertices in the graph.

Step 2: Select any vertex as starting point for traversal. Visit that vertex and push it on to the Stack.

Step 3: Visit any one of the adjacent vertex of the verex which is at top of the stack which is not visited and push it on to the stack.

Step 4: Repeat step 3 until there are no new vertex to be visit from the vertex on top of the stack.

Step 5: When there is no new vertex to be visit then use back tracking and pop one vertex from the stack.

Step 6: Repeat steps 3, 4 and 5 until stack becomes Empty.

Step 7: When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph

This function is best described recursively as in Program.

```
#define FALSE 0
#define TRUE 1
 int visited[MAX_VERTICES];
void dfs(int v)
{
  node_pointer w;
  visited[v]= TRUE;
  printf("%d", v);
  for (w=graph[v]; w; w=w->link)
    if (!visited[w->vertex])
      dfs(w->vertex);
}
```

Consider the graph G of Figure 6.16(a), which is represented by its adjacency lists as in Figure 6.16(b). If a depth-first search is initiated from vertex 0 then the vertices of G are visited in the following order: **0 1 3 7 4 5 2 6.** Since DFS(O) visits all vertices that can be reached from 0 the vertices visited, together with all edges in G incident to these vertices form a connected component of G.

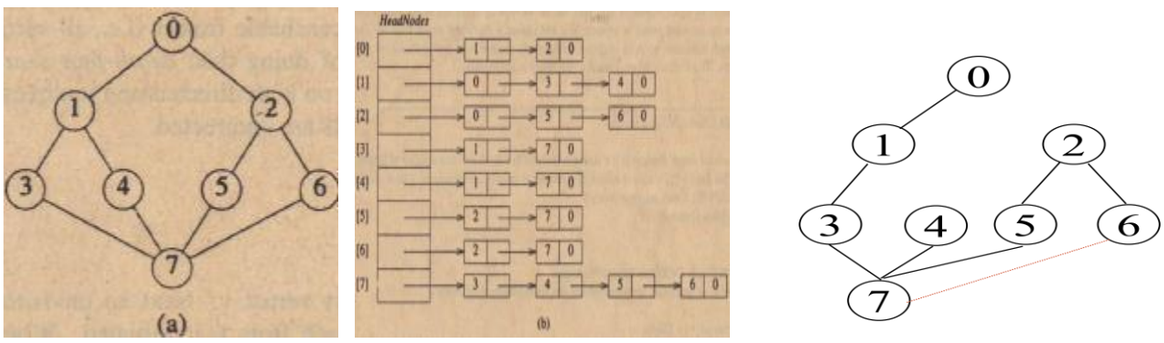


Figure: Graph and its adjacency list representation, DFS spanning tree

Analysis or DFS:
When G is represented by its adjacency lists, the vertices w adjacent to v can be determined by following a chain of links. Since DFS examines each node in the adjacency lists at most once and there are 2e list nodes the time to complete the search is O(e). If G is represented by its adjacency matrix then the time to determine all vertices adjacent to v is O(n). Since at most n vertices are visited the total time is $O(n^2)$.

Breadth-First Search
In a breadth-first search, we begin by visiting the start vertex v. Next all unvisited vertices adjacent to v are visited. Unvisited vertices adjacent to these newly visited vertices are then visited and so on. Algorithm BFS (Program 6.2) gives the details.

```
typedef struct queue *queue_pointer;
typedef struct queue {
   int vertex;
```

```
   queue_pointer link;
};
void addq(queue_pointer *,
       queue_pointer *, int);
int deleteq(queue_pointer *);
void bfs(int v)
{
 node_pointer w;
 queue_pointer front, rear;
 front = rear = NULL;
 printf("%d", v);
 visited[v] = TRUE;
 addq(&front, &rear, v);
while (front) {
   v= deleteq(&front);
   for (w=graph[v]; w; w=w->link)
     if (!visited[w->vertex]) {
     printf("%d", w->vertex);
       addq(&front, &rear, w->vertex);
       visited[w->vertex] = TRUE;
     }
 }
}
```

Steps:

BFS traversal of a graph, produces a spanning tree as final result. Spanning Tree is a graph without any loops. We use Queue data structure with maximum size of total number of vertices in the graph to implement BFS traversal of a graph.

We use the following steps to implement BFS traversal...

Step 1: Define a Queue of size total number of vertices in the graph.

Step 2: Select any vertex as starting point for traversal. Visit that vertex and insert it into the Queue.

Step 3: Visit all the adjacent vertices of the vertex which is at front of the Queue which is not visited and insert them into the Queue.

Step 4: When there is no new vertex to be visit from the vertex at front of the Queue then delete that vertex from the Queue.

Step 5: Repeat step 3 and 4 until queue becomes empty.

Step 6: When queue becomes Empty, then produce final spanning tree by removing unused edges from the graph

Analysis Of BFS:

Each visited vertex enters the queue exactly once. So the while loop is iterated at most n times If an adjacency matrix is used the loop takes O(n) time for each vertex visited. The total time is therefore, $O(n^2)$. If adjacency lists are used the loop has a total cost of $d_0 + \ldots + d_{n-1} = O(e)$, where d is the degree of vertex i. As in the case of DFS all visited vertices together with all edges incident to them, form a connected component of G.

3. Connected Components

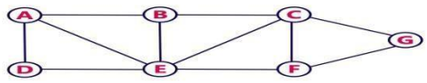
If G is an undirected graph, then one can determine whether or not it is connected by simply making a call to either DFS or BFS and then determining if there is any unvisited vertex. The connected components of a graph may be obtained by making repeated calls to either DFS(v) or BFS(v); where v is a vertex that has not yet been visited. This leads to function Connected(Program 6.3), which determines the connected components of G. The algorithm uses DFS (BFS may be used instead if desired). The computing time is not affected. Function connected –Output outputs all vertices visited in the most recent invocation of DFS together with all edges incident on these vertices.

```
void connected(void){
  for (i=0; i<n; i++) {
     if (!visited[i]) {
        dfs(i);
 printf("\n");        }     } }
```

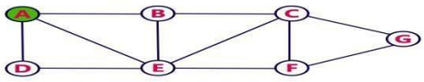
Analysis of Components:

If G is represented by its adjacency lists, then the total time taken by dfs is O(e). Since the for loops take O(n) time, the total time to generate all the Connected components is O(n+e). If adjacency matrices are used,then the time required is $O(n^2)$

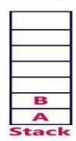
Consider the following example graph to perform DFS traversal



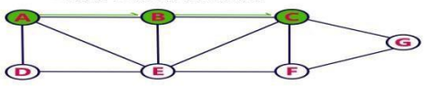
Step 1:
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.

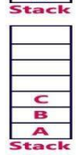


Step 2:
- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.

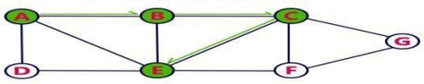


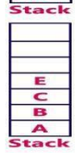
Step 3:
- Visit any adjacent vertext of **B** which is not visited (**C**).
- Push C on to the Stack.



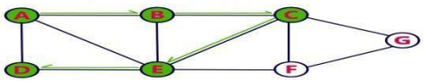


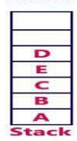
Step 4:
- Visit any adjacent vertext of **C** which is not visited (**E**).
- Push E on to the Stack



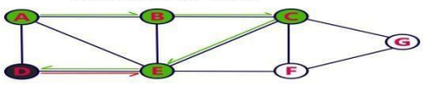


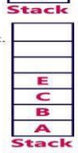
Step 5:
- Visit any adjacent vertext of **E** which is not visited (**D**).
- Push D on to the Stack



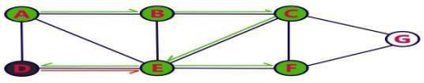


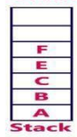
Step 6:
- There is no new vertiex to be visited from D. So use back track.
- Pop D from the Stack.



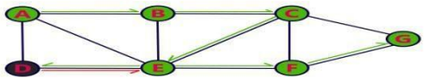


Step 7:
- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.

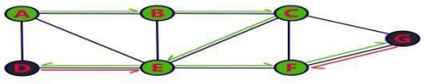


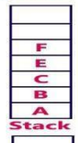


Step 8:
- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.

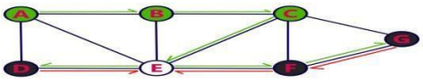


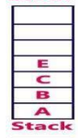
Step 9:
- There is no new vertiex to be visited from G. So use back track.
- Pop G from the Stack.



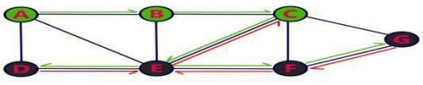


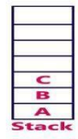
Step 10:
- There is no new vertiex to be visited from F. So use back track.
- Pop F from the Stack.



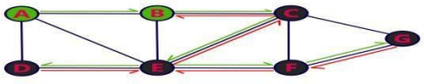


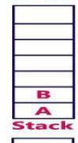
Step 11:
- There is no new vertiex to be visited from E. So use back track.
- Pop E from the Stack.



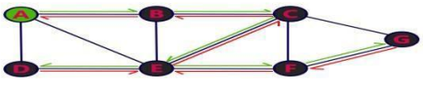


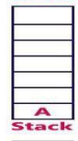
Step 12:
- There is no new vertiex to be visited from C. So use back track.
- Pop C from the Stack.



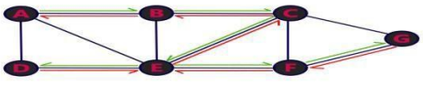


Step 13:
- There is no new vertiex to be visited from B. So use back track.
- Pop B from the Stack.

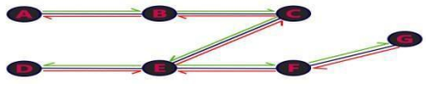




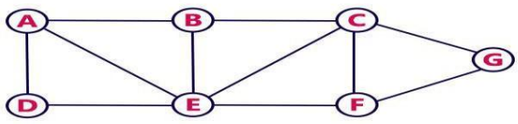
Step 14:
- There is no new vertiex to be visited from A. So use back track.
- Pop A from the Stack.



- Stack became Empty. So stop DFS Treversal.
- Final result of DFS traversal is following spanning tree.

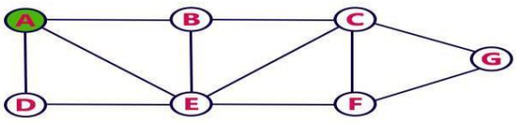


Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

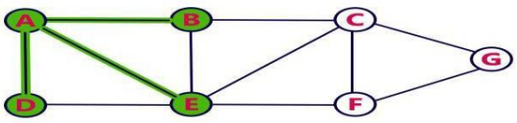


Queue

A						

Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

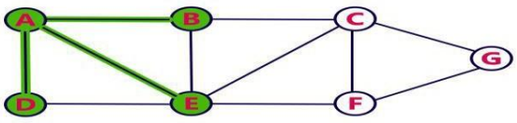


Queue

	D	E	B			

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

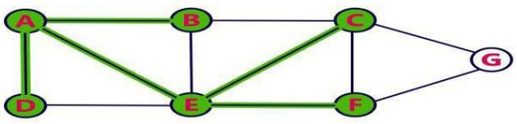


Queue

		E	B			

Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

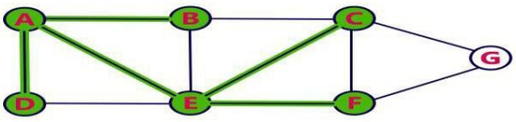


Queue

			B	C	F	

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

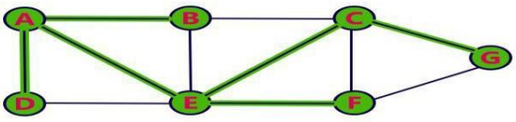


Queue

				C	F	

Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.

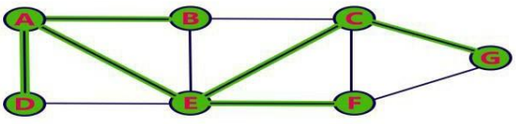


Queue

					F	G

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

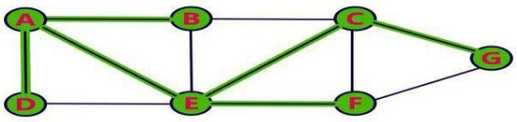


Queue

						G

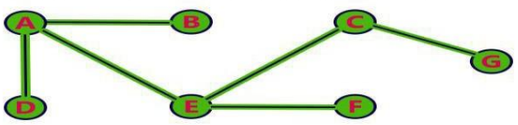
Step 8:

- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue

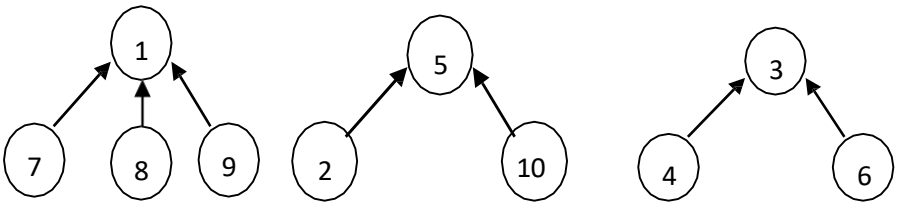
- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



Set Representation:

Sets and Disjoint Set Union:

Disjoint Set Union: Considering a set S={1,2,3…10} (when n=10), then elements can be partitioned into three disjoint sets s1={1,7,8,9},s2={2,5,10} and s3={3,4,6}. Possible tree representations are:



In this representation each set is represented as a tree. Nodes are linked from the child to parent rather than usual method of linking from parent to child.

The operations on these sets are:

1. Disjoint setunion
2. Find(i)
3. MinOperation
4. Delete
5. Intersect

1. Disjoint Set union:

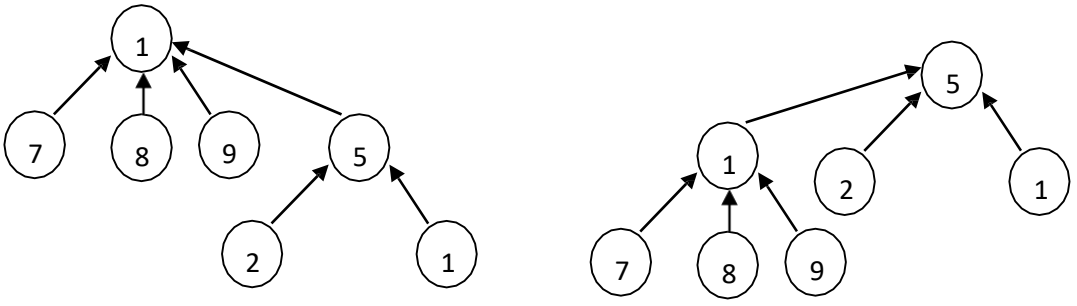
If S_i and S_j are two disjoint sets, then their union S_i U S_j = all the elements x such that x is in S_i or S_j. Thus S_1 U S_2 ={1,7,8,9,2,5,10}.

2. Find(i):

Given the element I, find the set containing i. Thus, 4 is in set S_3, 9 is in S_1.

UNION operation:

Union(i,j) requires two tree with roots i and j be joined. S_1 U S_2 is obtained by making any one of the sets as sub tree of other.



Simple Algorithm for Union:

```
Algorithm Union(i,j)
{
//replace the disjoint sets with roots i and j, I not equal to j by their
union Integer i,j;
P[j] :=i;
}
```

Example:

Implement following sequence of operations Union(1,3),Union(2,5),Union(1,2)

Solution:

Initially parent array contains zeros.

0	0	0	0	0	0
1	2	3	4	5	6

1. After performing union(1,3)operationParent[3]:=1

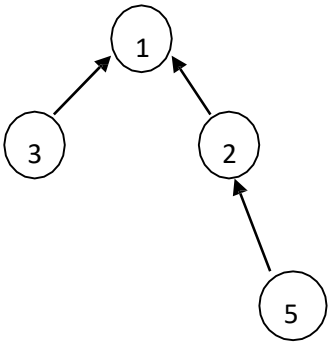
0	0	1	0	0	0
1	2	3	4	5	6

2. After performing union(2,5)operationParent[5]:=2

0	0	1	0	2	0
1	2	3	4	5	6

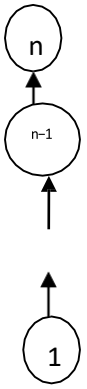
3. After performing union(1,2)operationParent[2]:=1

0	1	1	0	2	0
1	2	3	4	5	6



Process the following sequence of union operations Union(1,2),Union(2,3)Union(n-1,n)

Degenerate Tree:



The time taken for n-1 unions is O(n).

Find(i) operation: determines the root of the tree containing

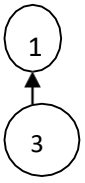
element i. Simple Algorithm for Find:

Algorithm Find(i)

```
{
    j:=i;
    while(p[j]>0) do
    j:=p[j]; return j;
}
```

Find Operation: Find(i) implies that it finds the root node of i^{th} node, in other words it returns the name of the set i.

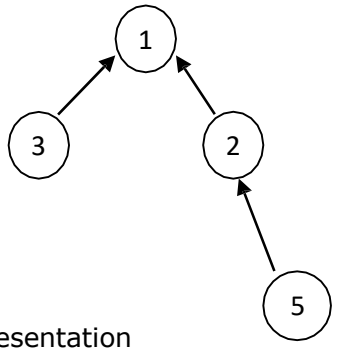
Example: Consider the Union(1,3)



Find(1)=0
Find(3)=1, since its parent is 1. (i.e, root is 1)

Example:

Considering



Array Representation

P[i]	0	1	1	2
i	1	2	3	5

Find(5)=1
Find(2)=1
Find(3)=1

The root node represents all the nodes in the tree. Time Complexity of „n“ find operations is $O(n^2)$.

To improve the performance of union and find algorithms by avoiding the creation of degenerate tree. To accomplish this, we use weighting rule for Union(i,j).

Weighting Rule for Union(i,j)

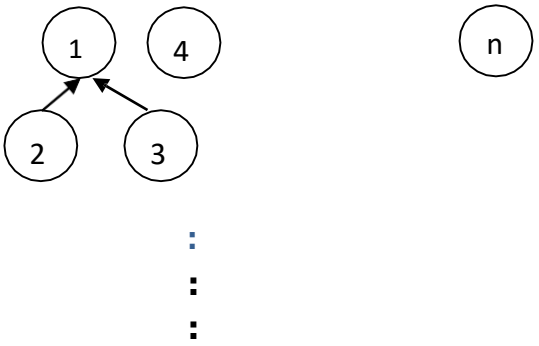
```
1   Algorithm WeightedUnion(i, j)
2   // Union sets with roots i and j, i ≠ j, using the
3   // weighting rule. p[i] = -count[i] and p[j] = -count[j].
4   {
5       temp := p[i] + p[j];
6       if (p[i] > p[j]) then
7       { // i has fewer nodes.
8           p[i] := j; p[j] := temp;
9       }
10      else
11      { // j has fewer or equal nodes.
12          p[j] := i; p[i] := temp;
13      }
14  }
```

Union algorithm with weighting rule

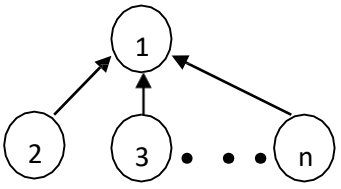
Tree obtained with
weighted Initially

Union(1,2)

Union(1,3)



Union(1,n)



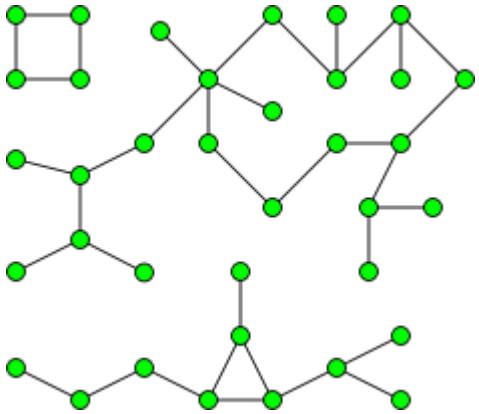
Collapsing Rule for Find(i)

```
1   Algorithm CollapsingFind(i)
2   // Find the root of the tree containing element i. Use the
3   // collapsing rule to collapse all nodes from i to the root.
4   {
5       r := i;
6       while (p[r] > 0) do r := p[r]; // Find the root.
7       while (i ≠ r) do  // Collapse nodes from i to root r.
8       {
9           s := p[i]; p[i] := r; i := s;
10      }
11      return r;
12  }
```

Find algorithm with collapsing rule

Connected components

In graph theory, a connected component (or just component) of an undirected graph is a subgraph in which any two vertices are connected to each other by paths, and which is connected to no additional vertices in the super graph. For example, the graph shown in the illustration has three connected components. A vertex with no incident edges is itself a connected component. A graph that is itself connected has exactly one connected component, consisting of the whole graph.



A graph with three connected components.

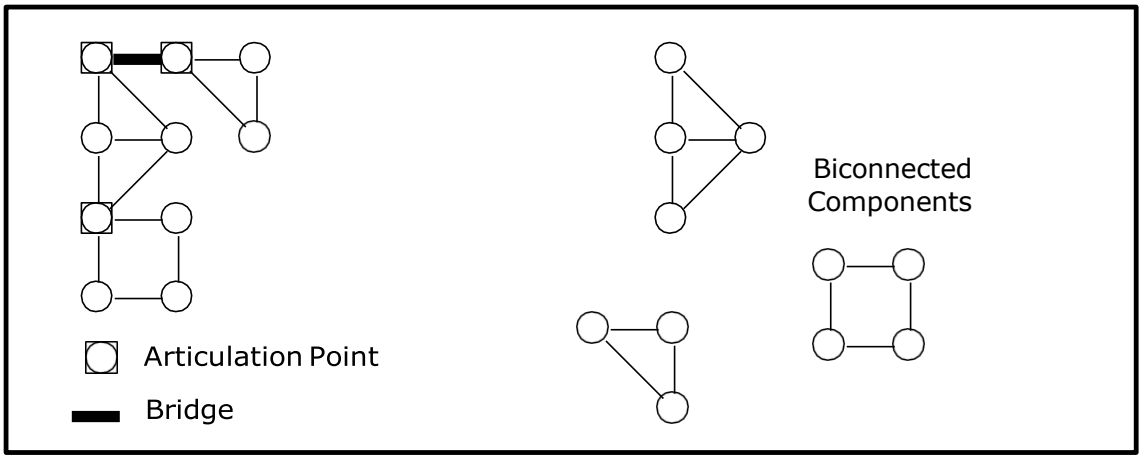
Biconnected Components:

Let G = (V, E) be a connected undirected graph. Consider the following definitions:

Articulation Point (or Cut Vertex): An articulation point in a connected graph is a vertex (together with the removal of any incident edges) that, if deleted, would break the graph into two or more pieces..

Bridge: Is an edge whose removal results in a disconnected graph.

Biconnected: A graph is biconnected if it contains no articulation points. In a biconnected graph, two distinct paths connect each pair of vertices. A graph that is not biconnected divides into biconnected components. This is illustrated in the following figure:



Articulation Points and Bridges

Biconnected graphs and articulation points are of great interest in the design of network algorithms,

because these are the "critical" points, whose failure will result in the network becoming disconnected.

Let us consider the typical case of vertex v, where v is not a leaf and v is not the root. Let $w_1, w_2, \ldots\ldots w_k$ be the children of v. For each child there is a subtree of the DFS tree rooted at this child. If for some child, there is no back edge going to a proper ancestor of v, then if we remove v, this subtree becomes disconnected from the rest of the graph, and hence v is an articulation point.

L (u) = min {DFN (u), min {L (w) | w is a child of u}, min {DFN (w) | (u, w) is a back edge}}.

L (u) is the lowest depth first number that can be reached from 'u' using a path of descendents followed by at most one back edge. It follows that, If 'u' is not the root then 'u' is an articulation point iff 'u' has a child 'w' such that:

L (w) ≥ DFN (u)

Algorithm for finding the Articulation points:

Pseudocode to compute DFN and L.

Algorithm Art (u, v)

```
// u is a start vertex for depth first search. V is its parent if any in the depth first
// spanning tree. It is assumed that the global array dfn is initialized to zero and
that // the global variable num is initialized to 1. n is the number of vertices in G.
{
        dfn [u] := num; L [u] := num; num
        := num + 1; for each vertex w
        adjacent from u do
        {
                if (dfn [w] = 0) then
                {
```

```
                    Art  (w,  u);                    // w
                    is unvisited. L [u] := min (L [u], L
                    [w]);
                }
        else if (w ≠ v) then L [u] := min (L [u], dfn [w]);
        }
}
```

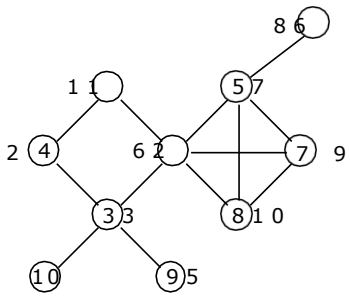
Algorithm for finding the Biconnected Components:

Algorithm BiComp (u, v)

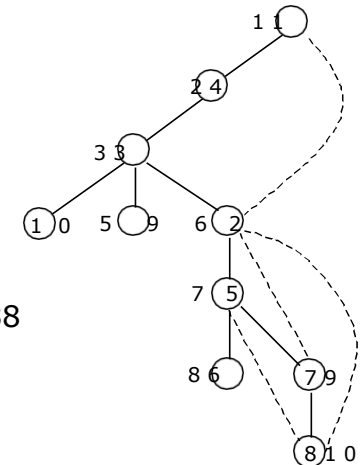
```
// u is a start vertex for depth first search. V is its parent if any in the depth first
// spanning tree. It is assumed that the global array dfn is initialized to 1. n is the number of vertices in G.
{
        dfn [u] := num; L [u] := num; num
        := num + 1; for each vertex w
        adjacent from u do
        {
                if ((v ≠ w) and (dfn [w] ≤ dfn
                        [u])) then add (u, w)
                        to the top of a stack
                        s;
                if (dfn [w] = 0) then
                {
                        if (L [w] ≥ dfn [u]) then
                        {
                                write ("New
                                bicomponent");
                                repeat
                                {
                                        Delete an edge from the
                                        top of stack s; Let this
                                        edge be (x, y);
                                  Write (x, y);
                                } until (((x, y) = (u, w)) or ((x, y) = (w, u)));
                        }
                        BiComp  (w,  u);     // w is unvisited. L [u] := min (L [u], L [w]);
                }
                else if (w ≠ v) then L [u] : = min (L [u], dfn [w]);
        }
}
```

Example:

For the following graph identify the articulation points and Biconnected components:



4



To identify the articulation points, we use:

L (u) = min {DFN (u), min {L (w) | w is a child of u}, min {DFN (w) | w is a vertex to which there is back edge from u}}

L (1) = min {DFN (1), min {L (4)}} = min {1, L (4)} = min {1, 1} = 1

L (4) = min {DFN (4), min {L (3)}} = min {2, L (3)} = min {2, 1} = 1

L (3) = min {DFN (3), min {L (10), L (9), L (2)}} =
= min {3, min {L (10), L (9), L (2)}} = min {3, min {4, 5, 1}} = 1

L (10) = min {DFN (10)} = 4

L (9) = min {DFN (9)} = 5

L (2) = min {DFN (2), min {L (5)}, min {DFN (1)}}
= min {6, min {L (5)}, 1} = min {6, 6, 1} = 1

L (5) = min {DFN (5), min {L (6), L (7)}} = min {7, 8, 6} = 6

L (6) = min {DFN (6)} = 8

L (7) = min {DFN (7), min {L (8}, min {DFN (2)}}
= min {9, L (8) , 6} = min {9, 6, 6} = 6

L (8) = min {DFN (8), min {DFN (5), DFN (2)}}
= min {10, min (7, 6)} = min {10, 6} = 6

Therefore, L (1: 10) = (1, 1, 1, 1, 6, 8, 6, 6, 5, 4)

Finding the Articulation Points:

Vertex 1: Vertex 1 is not an articulation point. It is a root node. Root is an articulation point if it has two or more child nodes.

Vertex 2: is an articulation point as child 5 has L (5) = 6 and DFN (2) = 6, So, the condition L (5) = DFN (2) is true.

Vertex 3: is an articulation point as child 10 has L (10) = 4 and DFN (3) = 3, So, the condition L (10) > DFN (3) is true.

Vertex 4: is not an articulation point as child 3 has L (3) = 1 and DFN (4) = 2, So, the condition L (3) $\geq$ DFN (4) is false.

Vertex 5: is an articulation point as child 6 has L (6) = 8, and DFN (5) = 7, So, the condition L (6) > DFN (5) is true.

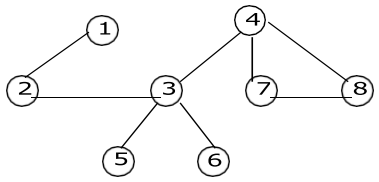
Vertex 7: is not an articulation point as child 8 has L (8) = 6, and DFN (7) = 9, So, the condition L (8) $\geq$ DFN (7) is false.

Vertex 6, Vertex 8, Vertex 9 and Vertex 10 are leaf nodes.

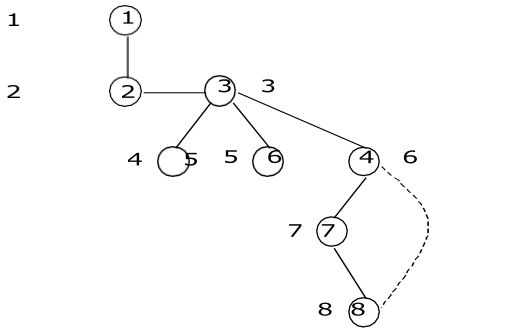
Therefore, the articulation points are {2, 3, 5}.

Example:

For the following graph identify the articulation points and Biconnected components:



Graph



DFS spanning Tree

L (u) = min {DFN (u), min {L (w) | w is a child of u}, min {DFN (w) | w is a vertex to which there is back edge from u}}

L (1) = min {DFN (1), min {L (2)}} = min {1, L (2)} = min {1, 2} = 1

L (2) = min {DFN (2), min {L (3)}} = min {2, L (3)} = min {2, 3} = 2

L (3) = min {DFN (3), min {L (4), L (5), L (6)}} = min {3, min {6, 4, 5}} = 3

L (4) = min {DFN (4), min {L (7)} = min {6, L (7)} = min {6, 6} = 6

L (5) = min {DFN (5)} = 4

L (6) = min {DFN (6)} = 5

L (7) = min {DFN (7), min {L (8)}} = min {7, 6} = 6

L (8) = min {DFN (8), min {DFN (4)}} = min {8, 6} = 6

Therefore, L (1: 8) = {1, 2, 3, 6, 4, 5, 6, 6}

Finding the Articulation Points:

Check for the condition if L (w) $\geq$ DFN (u) is true, where w is any

child of u. Vertex 1: Vertex 1 is not an articulation point.
It is a root node. Root is an articulation point if it has two or more child nodes.

Vertex 2: is an articulation point as L (3) = 3 and DFN (2) = 2.
So, the condition is true

Vertex 3: is an articulation Point as:
I. L (5) = 4 and DFN (3) = 3

II. L (6) = 5 and DFN (3) = 3 and
III. L (4) = 6 and

DFN (3) = 3 So, the

condition true in above

cases

Vertex 4: is an articulation point as L (7) = 6 and DFN (4) = 6.
So, the condition is true

Vertex 7: is not an articulation point as L (8) = 6 and DFN (7) = 7.
So, the condition is False

Vertex 5, Vertex 6 and Vertex 8 are leaf

nodes. Therefore, the articulation points

are {2, 3, 4}.